

B.3. Beispiel 3

```
1 import math as m
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5 from numpy import linalg as la
6
7
8 # Berechne Jacobi-Matrix
9 def jacobi(args, x, rel_stoffe):
10     k_h = args[0]
11     k_z = args[1]
12     sm = args[2]
13
14     jm = np.zeros((len(rel_stoffe), len(x)))
15
16     jm[0, 0] = -k_z[0] * x[18]
17     jm[0, 1] = k_h[0]
18     jm[0, 18] = -k_z[0] * x[0]
19
20     jm[1, 1] = -k_z[1] * x[18]
21     jm[1, 2] = k_h[1]
22     jm[1, 18] = -k_z[1] * x[1]
23
24     jm[2, 3] = -k_z[2] * x[18]
25     jm[2, 4] = k_h[2]
26     jm[2, 18] = -k_z[2] * x[3]
27
28     jm[3, 5] = -k_z[3] * x[18]
29     jm[3, 6] = k_h[3]
30     jm[3, 18] = -k_z[3] * x[5]
31
32     jm[4, 7] = -k_z[4] * x[18]
33     jm[4, 8] = k_h[4]
34     jm[4, 18] = -k_z[4] * x[7]
35
36     jm[5, 9] = -k_z[5] * x[18]
37     jm[5, 10] = k_h[5]
38     jm[5, 18] = -k_z[5] * x[9]
39
40     jm[6, 11] = -k_z[6] * x[18]
41     jm[6, 12] = k_h[6]
42     jm[6, 18] = -k_z[6] * x[11]
43
44     jm[7, 12] = -k_z[7] * x[18]
45     jm[7, 13] = k_h[7]
46     jm[7, 18] = -k_z[7] * x[12]
47
48     jm[8, 14] = -k_z[8] * x[18]
49     jm[8, 15] = k_h[8]
50     jm[8, 18] = -k_z[8] * x[14]
51
52     jm[9, 16] = -k_z[9] * x[18]
53     jm[9, 17] = k_h[9]
54     jm[9, 18] = -k_z[9] * x[16]
55
56     jm = np.matmul(sm, jm)
57
```

```

58     return jm
59
60
61 # Berechne Funktionswerte
62 def fkt_wert(args, x, rel_stoffe):
63     k_h = args[0]
64     k_z = args[1]
65     sm = args[2]
66
67     fw = np.zeros(len(rel_stoffe))
68
69     fw[0] = k_h[0] * x[1] - k_z[0] * x[0] * x[18]
70     fw[1] = k_h[1] * x[2] - k_z[1] * x[1] * x[18]
71     fw[2] = k_h[2] * x[4] - k_z[2] * x[3] * x[18]
72     fw[3] = k_h[3] * x[6] - k_z[3] * x[5] * x[18]
73     fw[4] = k_h[4] * x[8] - k_z[4] * x[7] * x[18]
74     fw[5] = k_h[5] * x[10] - k_z[5] * x[9] * x[18]
75     fw[6] = k_h[6] * x[12] - k_z[6] * x[11] * x[18]
76     fw[7] = k_h[7] * x[13] - k_z[7] * x[12] * x[18]
77     fw[8] = k_h[8] * x[15] - k_z[8] * x[14] * x[18]
78     fw[9] = k_h[9] * x[17] - k_z[9] * x[16] * x[18]
79
80     fw = np.matmul(sm, fw)
81
82     return fw
83
84 # Berechne Änderungsrate mit Vorkonditionierung
85 def delta_x(args, t, x, rel_stoffe):
86     i = np.eye(len(x))
87     jm = jacobi(args, x, rel_stoffe)
88     am = i - t * jm
89     tm = t_matrix(am)
90
91     ta = np.matmul(tm, am)
92     tat = np.matmul(ta, tm)
93     tat_inv = np.linalg.solve(tat, i)
94     tf = np.matmul(tm, fkt_wert(args, x, rel_stoffe))
95
96     y = np.matmul(tat_inv, tf)
97     dx = np.matmul(tm, y)
98
99     return dx
100
101
102 # Berechne optimalen Zeitschritt
103 def t_opt(args, t_n, x_n, x_n1, dx, rel_stoffe):
104     z = abs(np.matmul(np.transpose(dx), fkt_wert(args, x_n, rel_stoffe) - dx))
105     n = 2 * la.norm(dx) * la.norm(fkt_wert(args, x_n1, rel_stoffe) - dx)
106     return (z / n) * t_n
107
108 # Berechne SER Zeitschritt
109 def t_ser(args, t_n, x_n, x_n1, rel_stoffe):
110     z = t_n * la.norm(fkt_wert(args, x_n, rel_stoffe))
111     n = la.norm(fkt_wert(args, x_n1, rel_stoffe))
112     return z / n
113
114 # Berechne D Matrix zur Vorkonditionierung
115 def d_matrix(am):
116     dm = np.zeros((len(am), len(am)))

```

```

117     for i in range(0, len(am)):
118         dm[i, i] = 1 / max(abs(am[i, :]))
119     return dm
120
121
122 # Berechne T Matrix zur Vorkonditionierung
123 def t_matrix(am):
124     tm = np.zeros((len(am), len(am)))
125     for i in range(0, len(am)):
126         tm[i, i] = m.sqrt(1 / max(abs(am[i, :]))))
127     return tm
128
129
130 # Zeige df
131 def plot_df(df, stoffe):
132     pd.set_option('display.max_columns', 19)
133     print(df)
134
135     df.plot()
136     plt.show()
137     plt.xlabel('Iterationen')
138     plt.ylabel('Konzentrationen')
139     plt.legend(stoffe)
140
141     return
142
143
144 # Schreibe df als csv in das LaTeX-Verzeichnis
145 def write_df(df, path, file):
146     # Schreibe Gesamttabelle
147     with open(path + file, 'w') as tf:
148         tf.write(df.to_csv())
149
150     # Erzeuge drei Teiltabellen zur besseren Darstellung in LaTeX
151     a = ['$Dap$', '$Dap+$', '$Dap2+$', '$Pip$', '$Pip+$', '$Pip2+$']
152     b = ['$Dea$', '$Dea+$', '$Tris$', '$Tris+$', '$Imi$', '$Imi+$']
153     c = ['$Bts$', '$Bts+$', '$Ace-$', '$Ace$', '$Lac-$', '$Lac$']
154     d = ['Residuum', 'Zeitschrittweite']
155
156     dfa = df.loc[:, a]
157     dfb = df.loc[:, b]
158     dfc = df.loc[:, c]
159     dfd = df.loc[:, d]
160
161     with open(path + file[:-4] + 'a.csv', 'w') as tf:
162         tf.write(dfa.to_csv())
163     with open(path + file[:-4] + 'b.csv', 'w') as tf:
164         tf.write(dfb.to_csv())
165     with open(path + file[:-4] + 'c.csv', 'w') as tf:
166         tf.write(dfc.to_csv())
167     with open(path + file[:-4] + 'd.csv', 'w') as tf:
168         tf.write(dfd.to_csv())
169
170     return
171
172
173 # Berechne Beispiel
174 def calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps):
175     x_n = x0

```

```

176 x_list = [x0]
177 t_list = [t_n]
178 res_list = [la.norm(fkt_wert(args, x_n, rel_stoffe))]
179 i= 0
180 while i in range(0, max_iter) and la.norm(fkt_wert(args, x_n, rel_stoffe)) >
eps:
181     dx = delta_x(args, t_n, x_n, rel_stoffe)
182     x_n1 = x_n + t_n * dx
183
184     if la.norm(fkt_wert(args, x_n1, rel_stoffe)) > la.norm(fkt_wert(args, x_n,
rel_stoffe))):
185         print('Abbruch: Keine Residual-Reduktion')
186         break
187     if opt:
188         t_n1 = t_opt(args, t_n, x_n, x_n1, dx, rel_stoffe)
189     else:
190         t_n1 = t_ser(args, t_n, x_n, x_n1, rel_stoffe)
191     if t_n1 < t_n:
192         print('Abbruch: Maximaler Zeitschritt erreicht')
193         break
194     t_list = np.vstack((t_list, t_n1))
195     res_list = np.vstack((res_list, la.norm(fkt_wert(args, x_n1, rel_stoffe))))
196     x_list = np.vstack((x_list, x_n1))
197     x_n = x_n1
198     t_n = t_n1
199     i = i + 1
200
201 df = pd.DataFrame(data=x_list, columns=stoffe)
202 df['Residuum'] = res_list
203 df['Zeitschrittweite'] = t_list
204 df.index.name = 'Iterationen'
205
206 return df
207
208
209 def main():
210     # Vorgegebene Konstanten für das Problem
211     k_h = np.array([2.4e-8, 2.29e-6, 1.31e-6, 8.47e-6, 1.02e-4,
212                     3.28e-4, 1.86e-7, 4.6e-3, 1.75e-2, 1.38e-1])
213     k_z = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
214     sm = np.array([[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
215                   [-1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
216                   [0, -1, 0, 0, 0, 0, 0, 0, 0, 0],
217                   [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
218                   [0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
219                   [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
220                   [0, 0, 0, -1, 0, 0, 0, 0, 0, 0],
221                   [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
222                   [0, 0, 0, 0, -1, 0, 0, 0, 0, 0],
223                   [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
224                   [0, 0, 0, 0, 0, -1, 0, 0, 0, 0],
225                   [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
226                   [0, 0, 0, 0, 0, 0, -1, 1, 0, 0],
227                   [0, 0, 0, 0, 0, 0, 0, -1, 0, 0],
228                   [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
229                   [0, 0, 0, 0, 0, 0, 0, 0, -1, 0],
230                   [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
231                   [0, 0, 0, 0, 0, 0, 0, 0, 0, -1],
232                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])

```

```

233     args = [k_h, k_z, sm]
234
235     # Anfängliche Konzentrationen
236     x0 = np.array([10, 0, 0, 10, 0,
237                   10, 0, 10, 0, 10,
238                   0, 10, 0, 0, 0,
239                   10, 0, 10, 10 ** -0.5])
240
241     # Anfänglich gewählter Zeitschritt
242     t_n = 1
243
244     stoffe = [ '$Dap$', '$Dap+$', '$Dap2+$', '$Dea$', '$Dea+$',
245              '$Tris$', '$Tris+$', '$Imi$', '$Imi+$', '$Bts$',
246              '$Bts+$', '$Pip$', '$Pip+$', '$Pip2+$',
247              '$Ace-$', '$Ace$', '$Lac-$', '$Lac$', '$H+$' ]
248     rel_stoffe = [ 'Dap+', 'Dap2+', 'Dea+', 'Tris+', 'Imi+',
249                  'Bts+', 'Pip+', 'Pip2+', 'Ace', 'Lac' ]
250
251     max_iter = 50
252     eps = 1e-10
253     for opt in [True, False]:
254         path = "../ba_nv_latex/"
255         if opt:
256             print("Beispiel: 3 OPT")
257             file = 'Data_Bsp3.csv'
258             df = calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps)
259             dfo = df.loc[:, 'Residuum']
260         else:
261             print("\nBeispiel: 3 SER")
262             file = 'Data_Bsp3SER.csv'
263             df = calc_bsp(args, x0, t_n, stoffe, rel_stoffe, max_iter, opt, eps)
264             dfs = df.loc[:, 'Residuum']
265             plot_df(df, stoffe)
266             write_df(df, path, file)
267
268 if __name__ == "__main__":
269     main()

```